El recocido simulado, linea por linea

Que hace cada instruccion del optimizador que la interfaz corre en la etapa «3 · Semilla y recocido», explicado desde cero: primero para que sirve, despues el algoritmo con una funcion de juguete, y despues el codigo, bloque por bloque, con una corrida real al lado.

Como esta hecho este documento

- **El codigo no esta copiado**: cada recuadro se extrae del archivo fuente al generar el PDF, con los numeros de linea reales. Si manana alguien edita `sa.py`, el proximo PDF muestra lo editado, no esto.
- **Los numeros tampoco**: salen de correr el ajuste sobre la corrida **co60** (Co-60, A @ 1550 V) mientras se arma el documento. La traza de las primeras propuestas se reproduce con la misma semilla del generador aleatorio y se verifica contra lo que devolvio el recocido de verdad: **coincide**.
- El documento hermano, `docs/recocido_simulado.pdf`, discute otra cosa: el metodo, que dice el paper y que evidencia hay de que el resultado es el optimo global. Este explica el programa.

Un aviso sobre el modelo que se usa para explicar

Todo el documento trabaja con el modelo **desnudo de dos parametros** (D y σ_{rel}), porque un paisaje de dos dimensiones se puede dibujar y mirar. La calibracion de verdad agrega una tercera componente -el exceso de cuentas a bajo canal- y da otro numero: **D = 1.2512** en vez de **1.2230**. El algoritmo es el mismo, con un parametro mas; la seccion 8 lo muestra tal como lo corre la interfaz.

1. Donde vive el recocido

El recocido no lee archivos ni sabe de fisica. Es el anteultimo eslabon de una cadena que ya hizo todo el trabajo sucio: leer el archivo del digitalizador, integrar el area de cada pulso, armar el histograma con resta de fondo, y elegir automaticamente en que tramo del espectro conviene ajustar. Lo que recibe es un problema matematico limpio: **una funcion que devuelve un numero, y hay que encontrar donde ese numero es minimo**.



La cadena completa. El recuadro naranja es lo que explica este documento; los dos que le siguen viven en el mismo archivo y la interfaz los muestra en la misma pestana.

En la interfaz (`analisis/gui.py`) esto es la pestana **3 · Semilla y recocido**. La etapa anterior, **2 · NCC e intervalo**, es la que entrega las dos cosas que el recocido necesita para arrancar: la **ventana** de ajuste y la **semilla**. Para esta corrida: ventana **[533, 1696] mV·ns** con ρ = 0.97746, y semilla **D = 1.2303**, σ_{rel} = 0.090.

1.1 Los archivos que intervienen

archivo	que aporta

`analisis/modelo.py`	arma el **problema**: junta datos, plantilla y ventana, y expone la funcion de costo
`analisis/sa.py`	el **algoritmo**: temperatura inicial, recocido, pulido, multiarranque y barras de error
`analisis/calibracion.py`	la **receta**: define las cotas, corre N recocidos, se queda con el mejor y traduce a calibracion
`analisis/pipeline.py`	el **orquestador** que llaman las dos interfaces, para que no haya dos copias de la receta
`analisis/gui.py`	los **cuatro paneles** de la pestana 3, que dibujan lo que el recocido dejo grabado

2. El vocabulario minimo

Seis palabras alcanzan para leer todo lo que sigue. Vale la pena fijarlas antes, porque el algoritmo es corto y lo dificil no es el codigo sino el vocabulario.

palabra	que significa aca
parametro	un numero que no conocemos y queremos determinar. Aca son dos: **D**, la ganancia en mV·ns por keV, que es el resultado buscado; y $\boldsymbol{\sigma_{rel}}$, cuanto borronea el detector, como fraccion del canal del borde.
vector x	los parametros juntos, en orden fijo. Para el algoritmo el punto x = (1.2230, 0.1256) es una posicion, nada mas.
costo	un numero que mide cuan mal le va a ese x contra los datos. Aca es el χ^2. Cuanto mas chico, mejor describe el modelo lo medido.
paisaje	el valor del costo en cada punto del plano (D, σ_{rel}). Es una superficie con valles y lomas: buscar el minimo es buscar el fondo del valle más hondo.
minimo local	el fondo de UN valle. Un optimizador que solo baja termina ahi y no tiene forma de saber, desde adentro, si hay un valle mas hondo del otro lado de la loma.
minimo global	el fondo del valle mas hondo de todos. Es el que se busca.

3. Que numero se esta minimizando

Antes del algoritmo hay que ver que es la funcion de costo, porque es lo unico que el recocido conoce del problema. El modelo dice que el espectro medido tiene que verse asi:

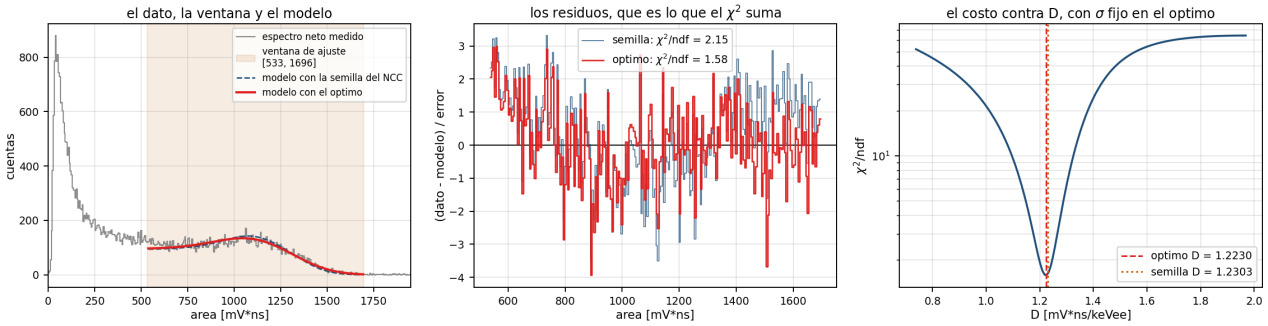
$$S(ch) \;=\; A \;\cdot\; \left[\, P\!\left(\frac{ch}{D}\right) \;\otimes\; G(\sigma) \,\right]$$

En palabras: se toma la **plantilla** P, que es el espectro que predice la simulacion de Geant4 en energia; se la lleva al eje medido dividiendo por **D** (que es justamente la ganancia que buscamos, en mV·ns por keV); se la borronea con una gaussiana de ancho **σ**, porque el detector no mide con precision infinita; y se la escala con una amplitud **A** para que tenga tantas cuentas como el histograma medido.

Comparar ese modelo con el dato bin por bin, pesando cada bin por su barra de error, da el χ^2:

$$\chi^2(\theta) \;=\; \sum_i \left(\frac{dato_i - modelo_i(\theta)}{error_i} \right)^2$$

Sobre esta corrida la ventana tiene **217 bins** y el modelo 3 parametros, o sea **214 grados de libertad**. Un ajuste que describe bien los datos da χ^2 parecido al numero de grados de libertad; por eso se reporta el cociente χ^2/ndf, que deberia dar alrededor de 1.



Izquierda: el espectro medido, la ventana que eligio el NCC y el modelo antes y despues de ajustar. Centro: los residuos, que es lo que el χ^2 eleva al cuadrado y suma. Derecha: el costo como funcion de D, con σ congelado. Ese corte es una rebanada del paisaje que el recocido recorre.

3.1 El costo, en el codigo

El optimizador llama siempre a la misma funcion, y es de tres lineas utiles. Lo que hace primero es rechazar lo imposible: un ancho negativo no significa nada, asi que en vez de calcular devuelve un numero enorme. Ese 1e30 es un **muro**: el algoritmo lo ve como un costo altisimo y no vuelve.

```
252  def costo(self, x):
253      """..."""
254      v = self.valores(x)
255      if not (v["D"] > 0 and v["sigma_rel"] > 0):
256          return 1e30
257      if self.exceso and not v["lam"] > 0:
258          return 1e30
259      return self.chi2(v["D"], v["sigma_rel"], v["lam"], c0=v["c0"])[0]
```

modelo.py · lineas 252 a 259

Y el χ^2 propiamente dicho. La linea `r = (self.dato - A * f) / self.error` es la formula de arriba escrita en numpy: resta vectorial, division vectorial, y despues la suma de los cuadrados.

```
219      self.n_eval += 1
220      f = self.forma(D, sigma_rel, c0)
221      if not self.exceso:
222          if A is None:
223              A = amplitud_optima(self.dato, self.error, f)
224          self.C_exceso = 0.0
225          r = (self.dato - A * f) / self.error
226          return float(np.sum(r * r)), float(A)
```

modelo.py · lineas 219 a 226

3.2 Un parametro que no hace falta buscar

El modelo tiene tres parametros -A, D y σ- pero el recocido busca en dos. La razon es que **A entra linealmente**: fijados D y σ, la amplitud que minimiza el χ^2 se despeja exacto, sin buscarla. Se llama proyeccion, o VARPRO, y en la practica compra dos cosas:

- una dimension menos que explorar, que es exponencialmente mas barato;
- no hay que elegir un paso comun para parametros de escalas distintas. Medido en esta corrida: A vale 0.04118 y D vale 1.2230, o sea que un mismo paso no le sirve a ninguno de los dos.

```
79  def amplitud_optima(dato, error, forma):
80      """..."""
90      w = 1.0 / np.maximum(error, 1e-9) ** 2
91      den = float(np.sum(w * forma * forma))
92      if den <= 0:
93          return 0.0
94      return float(np.sum(w * forma * dato) / den)
```

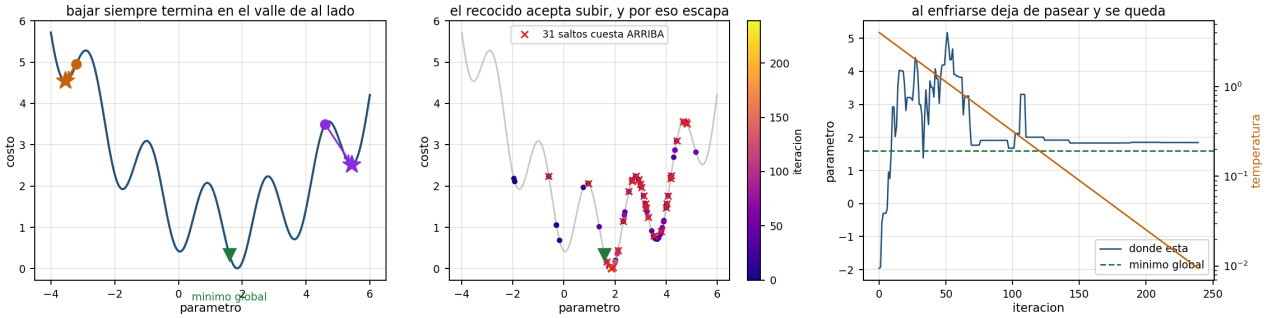
modelo.py · lineas 79 a 94

Con eso, el problema que ve el algoritmo es el mas simple posible: **una funcion de dos numeros que devuelve un numero**. Cuesta 68 microsegundos evaluarla, que es el dato que decide si un metodo que pide decenas de miles de evaluaciones es viable.

4. La idea, antes del codigo

El metodo obvio para minimizar es bajar: mirar alrededor, ir hacia donde el costo disminuye, repetir. Eso es un **optimizador local** y funciona perfecto mientras haya un solo valle. Cuando hay varios, termina en el que le toco por donde arranco, y no hay manera de darse cuenta desde adentro.

El recocido simulado cambia una sola regla: **a veces acepta moverse a un punto peor**. Esa es la unica forma de subir una loma y ver que hay del otro lado. Cuanto acepta lo controla un numero llamado **temperatura**: alta al principio, cuando conviene pasear; baja despues, cuando conviene afinar. El nombre viene de la metalurgia, donde enfriar despacio deja que los atomos encuentren la red de minima energia y enfriar de golpe los congela desordenados.



El algoritmo entero sobre una funcion de juguete de una variable. Izquierda: dos arranques, dos valles distintos, y ninguno es el bueno. Centro: el recorrido del recocido desde el mismo arranque malo; las cruces rojas son los movimientos que EMPEORARON el costo y aun asi fueron aceptados. Derecha: al bajar la temperatura la cadena deja de saltar y se queda.

4.1 La regla de aceptacion

Cada paso propone un punto nuevo y decide si se muda o se queda:

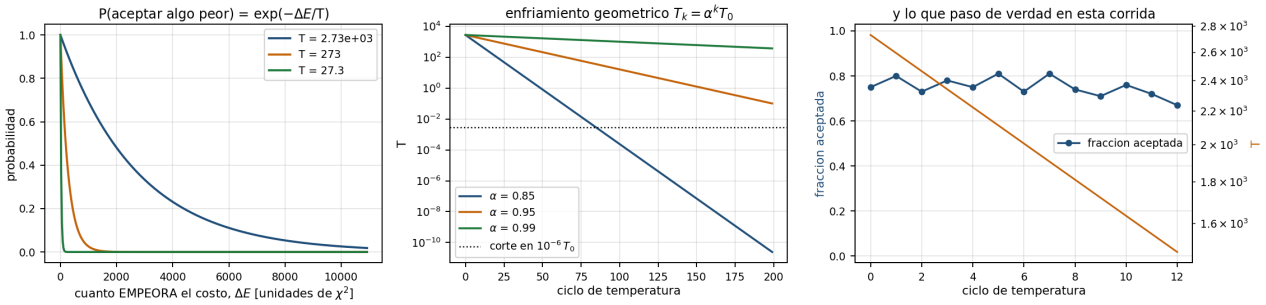
$$P(\text{aceptar}) = 1 \ \text{ si el costo baja;} \qquad P(\text{aceptar}) = e^{-\Delta E/T} \ \text{ si empeora en } \Delta E$$

Vale la pena leer la exponencial despacio, porque toda la mecanica del algoritmo esta ahi:

- si el empeoramiento ΔE es chico comparado con T, el exponente es casi cero y la probabilidad casi 1: se acepta casi siempre;
- si ΔE es grande comparado con T, la probabilidad se desploma: un salto muy malo casi nunca pasa;
- cuando T tiende a cero, ninguna subida se acepta y el metodo se vuelve un optimizador local corriente.

Lo importante de la comparacion es que **ΔE y T se miden en las mismas unidades**, las del costo. Un mismo valor de T puede ser enorme o ridiculo segun en que este medido el costo, y esa es la razon por la que el codigo calibra T en vez de copiarle el numero al paper (seccion 5.1).

El enfriamiento es geometrico: **T se multiplica por α = 0.95** despues de cada tanda de propuestas. Con eso T cae exponencialmente y el algoritmo pasa de explorar a afinar sin que haya que decirle cuando.



Izquierda: la regla de aceptacion para tres temperaturas, con la T que esta corrida calibro de verdad. Centro: el enfriamiento para los tres valores de α que menciona el paper. Derecha: la aceptacion medida ciclo a ciclo en la corrida real, que arranca en 75% y termina en 67%.

5. El codigo, bloque por bloque

Ahora si, `analisis/sa.py` de arriba abajo. Son cuatro funciones: calibrar la temperatura, recocer, pulir y repetir desde varios lados. Ninguna pasa de cuarenta lineas.

5.1 Elegir la temperatura inicial

El paper dice T_init = 2000 y no explica de donde sale. El problema es que T se compara contra ΔE, o sea que **2000 no significa nada hasta saber en que unidades esta el costo**. Con un χ^2 que vale ~460, T = 2000 anda bien; con un costo normalizado a 1, el mismo 2000 acepta absolutamente todo y la cadena se vuelve una caminata al azar que nunca converge.

La solucion del codigo es no elegir T sino **medirla**: se tiran unas cuantas propuestas al azar alrededor del punto de partida, se mira cuanto empeoran en promedio, y se despeja la T que hace que un empeoramiento tipico se acepte con la probabilidad que uno quiera.

```
69  def temperatura_inicial(costo, x0, delta, aceptacion=0.8, n=60, rng=None):
70      """..."""
82      rng = np.random.default_rng() if rng is None else rng
83      x0 = np.asarray(x0, float)
84      c0 = costo(x0)
85      subidas = []
86      for _ in range(n):
87          x = x0 + rng.uniform(-1, 1, size=x0.size) * delta
88          d = costo(x) - c0
89          if d > 0:
90              subidas.append(d)
91      if not subidas:
92          return max(abs(c0) * 1e-3, 1e-9)
93      return float(-np.mean(subidas) / np.log(aceptacion))
```

sa.py · lineas 69 a 93

- **linea 84**: se evalua el costo en el punto de partida, para tener contra que comparar.
- **lineas 86 a 90**: 60 propuestas al azar; de cada una se guarda cuanto EMPEORO, si es que empeoro. Las que mejoran no sirven para esto: se aceptan siempre, sin importar T.
- **linea 93**: se despeja T de exp(-<ΔE>/T) = aceptacion. Con la aceptacion objetivo del 80% que trae por defecto, esta corrida dio **T_init = 2729**, en unidades de χ^2.

5.2 Los argumentos del recocido

La funcion principal tiene muchos argumentos, pero casi todos son constantes del algoritmo con un valor por defecto sensato. Los tres primeros son el problema; el resto es como buscarlo.

argumento	por defecto	que es
`costo`	`requerido`	la funcion a minimizar. Recibe el vector de parametros y devuelve un numero. Al algoritmo no le importa que hay adentro.

`x0`	`requerido`	punto de partida. Aca, la semilla que entrego el NCC.
`cotas`	`requerido`	(minimo, maximo) de cada parametro. Acotan la busqueda a lo fisicamente posible y, de paso, fijan la escala del paso.
`delta`	`None`	cuanto se mueve como maximo cada parametro en una propuesta. Por defecto, el 10% del ancho de sus cotas: es el 0.1*d_i del paper.
`T_init`	`'auto'`	temperatura inicial. 'auto' la calibra midiendo el costo, que es lo unico que la hace independiente de las unidades.
`alfa`	`0.95`	cuanto se enfria por ciclo: T_(k+1) = alfa * T_k. El paper usa 0.95 y recomienda el rango 0.85 a 0.99.
`n_markov`	`100`	propuestas por temperatura. Las 100 del paper.
`T_min_rel`	`1e-06`	corte por temperatura, como fraccion de la inicial.
`n_estable`	`10`	cuantos ciclos seguidos sin mejorar hacen falta para cortar.
`tol_rel`	`0.0001`	que tan poco es 'no mejorar', en relativo. Relativo y no absoluto porque un corte absoluto depende de las unidades del costo, igual que T.
`paso_adaptativo`	`False`	agranda o achica el paso para mantener la aceptacion cerca de un objetivo. Apagado: sobre este problema no cambia nada y el default del paper es paso fijo.
`objetivo_aceptacion`	`0.35`	la aceptacion que persigue el paso adaptativo.
`semilla`	`None`	semilla del generador aleatorio. Fijarla hace que la corrida sea reproducible bit a bit.
`max_ciclos`	`400`	tope duro de ciclos de temperatura, por si ningun corte se cumple.

5.3 La preparacion

```
 99  def recocido(costo, x0, cotas, delta=None, T_init="auto", alfa=0.95,
100               n_markov=100, T_min_rel=1e-6, n_estable=10, tol_rel=1e-4,
101               paso_adaptativo=False, objetivo_aceptacion=0.35, semilla=None,
102               max_ciclos=400):
103      """..."""
129      rng = np.random.default_rng(semilla)
130      x = np.asarray(x0, float).copy()
131      cotas = np.asarray(cotas, float)
132      lo, hi = cotas[:, 0], cotas[:, 1]
133      if delta is None:
134          delta = 0.1 * (hi - lo)
135      delta = np.asarray(delta, float).copy()
136
137      T = (temperatura_inicial(costo, x, delta, rng=rng)
138           if T_init == "auto" else float(T_init))
139      T0 = T
140      escala = 1.0
141
142      c = costo(x)
143      c_inicial = c
144      mejor_x, mejor_c = x.copy(), c
145      historia = []
146      historia_x = []
147      historia_mejor_x = []
148      estables = 0
149      n_eval = 1
```

sa.py · lineas 99 a 149

- **linea 129**: el generador aleatorio. Al fijarle una semilla, la corrida entera es **reproducible**: los mismos numeros al azar, en el mismo orden, siempre. Es lo que permite volver a mirar una corrida que dio raro.
- **lineas 133 y 134**: el paso. Si no se lo dan, cada parametro se mueve como maximo un 10% del ancho de sus cotas. Para esta corrida eso es 0.1230 en D y 0.0280 en σ_{rel}.

- **linea 137**: la temperatura inicial, calibrada como se vio recien salvo que se la pasen a mano.
- **lineas 142 a 144**: la contabilidad. `x` y `c` son donde esta la cadena y cuanto vale ahi; `mejor_x` y `mejor_c` son el mejor punto visto hasta el momento. Esa separacion es clave y se explica sola en cuanto uno ve el bucle: la cadena puede irse a un lugar peor, pero el mejor punto no se pierde nunca.

5.4 El corazon: los dos bucles

Estas dieciocho lineas son el algoritmo. El bucle de afuera cuenta **ciclos de temperatura**; el de adentro hace las 100 propuestas que corresponden a esa temperatura.

```
151     for ciclo in range(max_ciclos):
152         aceptados = 0
153         for _ in range(n_markov):
154             prop = x + rng.uniform(-1, 1, size=x.size) * delta * escala
155             prop = np.clip(prop, lo, hi)
156             cp = costo(prop)
157             n_eval += 1
158             dE = cp - c
159             # Metropolis: si mejora se acepta siempre; si empeora, con exp(-dE/T).
160             if dE <= 0 or rng.random() < np.exp(-dE / max(T, 1e-300)):
161                 x, c = prop, cp
162                 aceptados += 1
163                 if c < mejor_c:
164                     mejor_x, mejor_c = x.copy(), c
165         tasa = aceptados / n_markov
166         historia.append((T, c, mejor_c, tasa, escala))
167         historia_x.append(x.copy())
168         historia_mejor_x.append(mejor_x.copy())
```

sa.py · lineas 151 a 168

- **linea 154** - **la propuesta**. Se toma el punto actual y se le suma un desplazamiento al azar, uniforme entre -δ y +δ en cada parametro. Es un salto a ciegas: nadie mira la pendiente, no hay derivadas en ningun lado. Por eso el metodo funciona con costos que no son derivables.
- **linea 155** - **las cotas**. `np.clip` recorta la propuesta al rectangulo permitido. Es la unica barrera fisica del algoritmo: sin esto la busqueda se puede ir a un σ negativo o a una D absurda.
- **lineas 156 y 158** - **cuanto cambia**. Se evalua el costo en la propuesta y se calcula ΔE. Negativo quiere decir que la propuesta es MEJOR.
- **linea 160** - **la regla de Metropolis**, la unica linea que distingue este metodo de una busqueda al azar. `dE <= 0` acepta toda mejora; si no, se sortea un numero entre 0 y 1 y se lo compara con exp(-ΔE/T). Python evalua el `or` de izquierda a derecha y corta apenas puede, asi que **cuando la propuesta mejora ni siquiera se sortea**.
- **linea 160** - el `max(T, 1e-300)` evita una division por cero cuando la temperatura ya se enfrio del todo. Detalle de implementacion, no de metodo.
- **lineas 161 y 162** - **mudarse**. Si se acepta, la cadena pasa a vivir en la propuesta. Notar que esto pasa tambien cuando el costo empeoro: es exactamente lo que se queria.
- **lineas 163 y 164** - **el mejor no se pierde**. Antes de seguir se compara con el record y, si es mejor, se guarda una COPIA. La copia importa: sin `.copy()` se guardaria una referencia al mismo array que el bucle sigue modificando.
- **lineas 165 a 168** - **la libreta**. Al cerrar cada ciclo se anota la temperatura, el costo actual, el mejor costo, la fraccion aceptada, y ademas DONDE estaba la cadena. Esa ultima columna es la que despues dibuja el camino en la pestana 3: sin ella se sabria que el costo bajo, pero no si para eso hubo que cruzar de un valle a otro.

5.5 Cuando parar

Hay dos cortes y hacen falta los dos. El primero mira si el resultado dejo de mejorar; el segundo, si la temperatura ya es tan baja que no puede pasar nada nuevo.

```
185         # corte por estabilizacion (relativo, no absoluto)
186         if len(historia) > 1:
187             prev = historia[-2][2]
188             if abs(prev - mejor_c) <= tol_rel * max(abs(prev), 1e-12):
189                 estables += 1
190             else:
191                 estables = 0
192             if estables >= n_estable:
193                 break
194 
195         T *= alfa
196         if T < T_min_rel * T0:
197             break
```

sa.py · lineas 185 a 197

- **lineas 186 a 193** - **corte por estabilizacion**. Si el mejor costo mejoro menos que `tol_rel` respecto del ciclo anterior, se cuenta un ciclo estable; si mejoro, el contador vuelve a cero. Con 10 ciclos estables seguidos, se corta. Es RELATIVO y no absoluto por lo mismo que la temperatura: un umbral absoluto depende de las unidades del costo, y el 0.001 del paper no sobrevive a cambiar de costo.
- **lineas 195 y 196** - **enfriar y corte por temperatura**. T se multiplica por α y, si cayo por debajo de una millonesima de la inicial, se termina. En esta corrida gano el corte por **estabilizacion**, a los **13 ciclos** y **1301 evaluaciones** del costo, en **0.11 segundos**.

5.6 Lo que devuelve

Devuelve **mejor_x**, no donde quedo la cadena. Es una distincion que se pasa por alto y cambia el resultado: la ultima propuesta aceptada puede ser peor que el record, porque el metodo acepta empeorar.

```
199     return dict(x=mejor_x, costo=mejor_c, historia=np.array(historia),
200                 historia_x=np.array(historia_x),
201                 historia_mejor_x=np.array(historia_mejor_x),
202                 x_inicial=np.asarray(x0, float).copy(),
203                 costo_inicial=float(c_inicial), delta=delta.copy(),
204                 n_eval=n_eval, T_init=T0, ciclos=len(historia),
205                 motivo=("estabilizacion" if estables >= n_estable
206                         else "temperatura minima"))
```

sa.py · lineas 199 a 206

5.7 Pulir

El recocido encuentra el valle correcto pero se detiene con la precision de su ultimo paso, que no es cero. El pulido es un optimizador local corriente que baja al fondo exacto desde ahi. Los dos hacen falta: el recocido solo da los primeros decimales, el pulido solo da el valle equivocado. En esta corrida el pulido movio el χ^2 de 338.1059 a 337.2826 y D de 1.22556 a 1.22303.

```
209 def pulir(costo, x, cotas=None):
210     """..."""
218     from scipy.optimize import minimize
219     r = minimize(costo, np.asarray(x, float), method="Nelder-Mead",
220                  options=dict(xatol=1e-7, fatol=1e-4, maxiter=4000))
221     return r.x, float(r.fun)
```

sa.py · lineas 209 a 221

Un detalle que conviene tener presente: `minimize` corre **sin cotas**. El argumento `cotas` de esta funcion se acepta y no se usa, asi que el pulido puede terminar afuera del rectangulo en el que el recocido tenia prohibido entrar. En la practica no molesta -el pulido arranca ya adentro del valle bueno y se mueve poco- pero es una asimetria real entre las dos etapas.

5.8 Correr muchas veces: el multiarranque

El recocido usa numeros al azar, asi que **una corrida no es un resultado, es una muestra**. Lo que se hace es correrlo N veces desde puntos iniciales **dispersos** y comparar. Si N busquedas independientes, arrancando de lugares distintos, terminan todas en el mismo punto, eso es evidencia practica de que ese punto es el minimo global. No es una demostracion, pero es lo mejor que se puede tener sin explorar todo el espacio.

```
224 def multiarranque(costo, cotas, n=8, x0=None, dispersion=0.25, semilla=0,
225                   callback=None, **kw):
226     """..."""
234     rng = np.random.default_rng(semilla)
235     cotas = np.asarray(cotas, float)
236     salidas = []
237     for i in range(n):
238         if callback is not None:
239             # el avance se cuenta por ARRANQUE y no por ciclo de temperatura:
240             # cuantos ciclos va a hacer cada recocido no se sabe de antemano
241             # (corta por estabilizacion), asi que inventar ese denominador daria
242             # una barra que retrocede.
243             callback(i / n, "arranque %d de %d" % (i + 1, n))
244         if x0 is None:
245             xi = rng.uniform(cotas[:, 0], cotas[:, 1])
246         else:
247             xi = np.asarray(x0, float) * (1 + dispersion *
248                                           rng.uniform(-1, 1, size=len(x0)))
249             xi = np.clip(xi, cotas[:, 0], cotas[:, 1])
250         r = recocido(costo, xi, cotas, semilla=int(rng.integers(1 << 30)), **kw)
251         r["x_pulido"], r["costo_pulido"] = pulir(costo, r["x"])
252         r["x0"] = xi
253         salidas.append(r)
254     return salidas
```

sa.py · lineas 224 a 254

- **linea 245**: sin x0, cada arranque sale de un punto **uniforme dentro de las cotas**. Es deliberado: si todos arrancaran de la semilla, que todos terminen igual no probaria nada.
- **linea 250**: cada corrida recibe su propia semilla, sorteada por el generador de arriba. Asi las N son independientes entre si y el conjunto sigue siendo reproducible.
- **linea 251**: cada corrida se pule antes de compararla, si no se estarian comparando resultados a medio terminar.

5.9 De donde salen las barras de error

El optimo por si solo no dice nada sobre su incerteza. Esa sale de la **curvatura** del costo en el minimo: si el valle es angosto, moverse poco empeora mucho el ajuste y el parametro esta bien determinado; si es ancho y plano, no. La funcion calcula esa curvatura numericamente -evaluando el costo alrededor del optimo- y la invierte para obtener la matriz de covarianza.

```
285     x = np.asarray(x, float)
286     n = x.size
287     if paso is not None:
288         h = np.abs(np.asarray(paso, float))
289     else:
290         h = np.maximum(np.abs(x) * paso_rel, 1e-9)
291     H = np.empty((n, n))
292     c0 = costo(x)
293     for i in range(n):
294         for j in range(n):
295             xp, xm = x.copy(), x.copy()
296             if i == j:
297                 xp[i] += h[i]; xm[i] -= h[i]
298                 H[i, i] = (costo(xp) - 2 * c0 + costo(xm)) / h[i] ** 2
299             else:
300                 a = x.copy(); a[i] += h[i]; a[j] += h[j]
301                 b = x.copy(); b[i] += h[i]; b[j] -= h[j]
302                 cc = x.copy(); cc[i] -= h[i]; cc[j] += h[j]
303                 d = x.copy(); d[i] -= h[i]; d[j] -= h[j]
304                 H[i, j] = (costo(a) - costo(b) - costo(cc) + costo(d)) / (4 * h[i] * h[j])
```

```
    try:
        C = 2.0 * np.linalg.inv(H)
        err = np.sqrt(np.maximum(np.diag(C), 0.0)) * np.sqrt(delta_chi2)
        return err, C
    except np.linalg.LinAlgError:
        return np.full(n, np.nan), np.full((n, n), np.nan)
```

sa.py · lineas 285 a 310

Sobre esta corrida da **D = 1.2230 ± 0.0034** (0.3%) y **σ_{rel} = 0.1256 ± 0.0036**, con una correlacion entre los dos de **-0.70**. Esa correlacion no es un detalle: dice que D y la resolucion se compensan entre si, asi que hay que reportarlas juntas.

6. Una corrida de verdad, numero por numero

Hasta aca el algoritmo en abstracto. Esta seccion es la misma corrida mirada por dentro: las primeras doce propuestas con todas sus cuentas a la vista. Los numeros no estan tipeados ni son un ejemplo inventado: salen de repetir las mismas lineas del bucle con la misma semilla del generador, y se verifican contra lo que devolvio el recocido (coincide el costo, la temperatura y la tasa de aceptacion al cerrar el primer ciclo).

	propuesta (D, sigma)	chi2	dE	P = exp(-dE/T)	sorteo	que hace
1	(1.2516, 0.1090)	406.3	-53.7	mejora	no sortea	acepta
2	(1.3073, 0.1015)	1202.1	+795.8	0.747	0.448	acepta
3	(1.2747, 0.0796)	583.6	-618.6	mejora	no sortea	acepta
4	(1.2017, 0.0675)	946.2	+362.7	0.876	0.314	acepta
5	(1.1557, 0.0718)	1658.2	+712.0	0.770	0.972	rechaza
6	(1.2693, 0.0838)	533.1	-413.2	mejora	no sortea	acepta
7	(1.3331, 0.0892)	1802.4	+1269.4	0.628	0.918	rechaza
8	(1.3160, 0.0838)	1261.0	+727.9	0.766	0.077	acepta
9	(1.3131, 0.0677)	1140.1	-120.8	mejora	no sortea	acepta
10	(1.2227, 0.0681)	710.9	-429.2	mejora	no sortea	acepta
11	(1.2929, 0.0566)	835.1	+124.1	0.956	0.769	acepta
12	(1.2992, 0.0369)	1012.4	+177.4	0.937	0.965	rechaza

Vale la pena leer tres filas de esa tabla:

- una propuesta que **mejora** se acepta sin sortear nada, y en la columna del sorteo dice justamente eso;
- una propuesta que **empeora** tiene una probabilidad calculada y se la compara con un numero al azar. A esta temperatura (T = 2729, del orden del propio χ^2) esa probabilidad es alta y varias subidas pasan;
- las que se rechazan no mueven la cadena: el punto actual sigue siendo el de antes, y en la fila siguiente la propuesta se construye otra vez desde ahi.

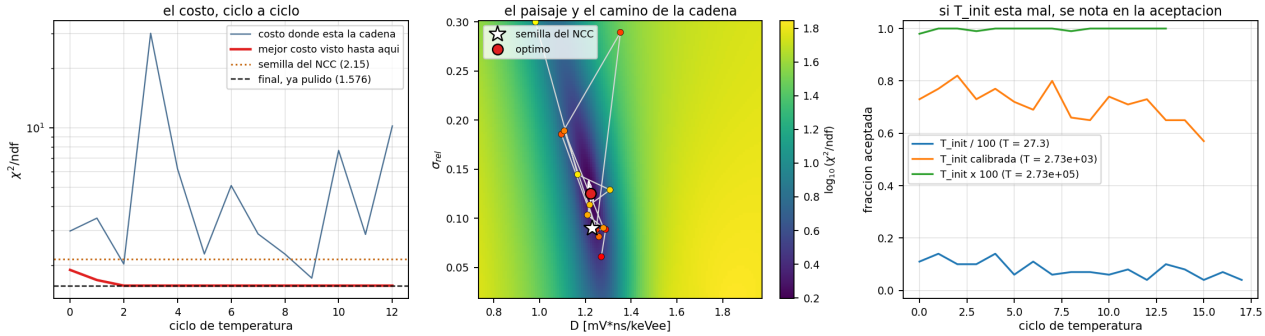
Sumadas las 100 propuestas del primer ciclo, se aceptaron el **75%**. Esa es la fraccion de aceptacion que la interfaz grafica, y es el termometro del algoritmo: alta al principio, casi cero al final.

6.1 Los ciclos, de principio a fin

ciclo	T	costo actual	mejor costo	aceptacion
1	2729	639.88	406.31	75 %
2	2593	743.95	360.93	80 %
3	2463	435.39	338.11	73 %

4	2340	6450.70	338.11	78 %
5	2223	1325.75	338.11	75 %
6	2112	489.41	338.11	81 %
7	2006	1088.28	338.11	73 %
8	1906	618.74	338.11	81 %
9	1811	489.04	338.11	74 %
10	1720	369.16	338.11	71 %
11	1634	1640.81	338.11	76 %
12	1552	615.03	338.11	72 %
13	1475	2184.75	338.11	67 %

Se ve el algoritmo entero en una tabla: la temperatura baja un 5% por fila, la aceptacion cae con ella, el costo actual deja de saltar y el mejor costo se aplana. Cuando se aplana durante 10 ciclos seguidos, corta.



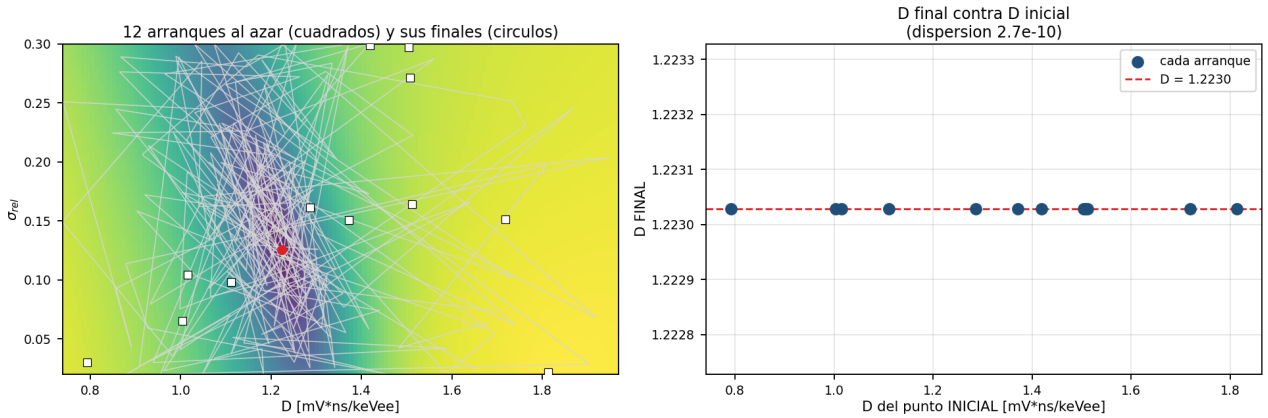
Izquierda: el costo ciclo a ciclo. La linea fina es donde esta la cadena -sube y baja, porque acepta empeorar- y la gruesa es el record, que solo puede bajar. Centro: el paisaje real del χ^2 y el camino que recorrio la cadena; las lineas blancas son los contornos de 1, 2 y 3 sigma. Derecha: que pasa con la aceptacion si la temperatura inicial esta cien veces mal en cada direccion.

7. Como se sabe que ese es el minimo

Un optimizador siempre devuelve algo. La pregunta es si ese algo es el minimo global o el fondo del primer pozo que encontro, y hay tres controles independientes.

7.1 Multiarranque

12 recocidos desde puntos al azar dentro de las cotas. Los 12 terminan en el mismo χ^2 (dentro de 0.1) y la dispersion de las D finales es **2.7e-10** mV·ns/keVee. Para comparar, la barra de error estadistica de D es 0.0034, o sea **1e+07 veces mas grande**: la incerteza del resultado no viene del optimizador.



Izquierda: los 12 arranques (cuadrados blancos), sus caminos y sus finales (circulos rojos), sobre el paisaje. Derecha: la D final contra la D del punto de partida. Que la nube sea plana es el resultado: de donde se arranque no cambia donde se termina.

7.2 Contra un optimizador local

El otro control es comparar con el metodo simple. Se resolvio el mismo problema desde **40 puntos de partida al azar**, una vez con un optimizador local corriente (Nelder-Mead, el mismo que usa el pulido) y otra con el recocido:

metodo	llegan al minimo	peor resultado
optimizador local, sin cotas	38 de 40	D = 10.242 con χ^2 = 10638
recocido + pulido	40 de 40	-

El resultado hay que leerlo con honestidad: en **este** paisaje el optimizador local casi siempre acierta, porque la region de interes tiene un solo valle. O sea que el recocido **no cambia el numero**, y eso es justamente lo que se queria saber. Lo que aporta es no depender de la suerte: el caso que fallo se escapo a D = 10.24 con un χ^2 32 veces peor, y nada en el codigo lo hubiera marcado como sospechoso si se corriera una sola vez.

> Notar la asimetria: el optimizador local corre sin cotas y el recocido recorta cada propuesta. Parte de la ventaja es del algoritmo y parte de haberle dicho donde puede buscar.

7.3 El paisaje

El tercer control es mirar. El panel del centro de la figura anterior es el χ^2 en todo el rectangulo de busqueda: si hubiera un segundo valle comparable, se veria. Y el minimo tiene que verse aproximadamente **cuadratico**, porque de esa suposicion salen las barras de error de la seccion 5.9. Si el valle fuera torcido o tuviera una meseta, la curvatura local mentiria y habria que reportar el error de otra manera.

8. Como lo llama la interfaz

El modulo `sa.py` no sabe nada de bordes Compton. Quien lo conecta con la fisica es `calibracion.calibrar`, que decide las cotas, corre el multiarranque y se queda con la mejor corrida.

```
166      cotas = [(0.6 * semilla[0], 1.6 * semilla[0]), (0.02, 0.30)]
167      if exceso:
168          # lambda: escala de caida del exceso [mV*ns]. La cota superior la fija
169          # el ancho de la ventana: un exceso que decae mas lento que eso es
170          # indistinguible de un fondo plano y se lo come la plantilla.
171          cotas.append((10.0, max(0.5 * (p.hi - p.lo), 50.0)))
172          semilla = tuple(semilla) + (0.15 * (p.hi - p.lo),)
173      if ordenada:
174          # c0 se acota a +-40% del borde: mas que eso no es un pedestal, es otro
175          # ajuste. La semilla es 0, o sea la hipotesis que se esta testeando.
```

```
176          B0 = semilla[0] * fisica.e_ref(rutas.resolver(corrida).fuente)
177          cotas.append((-0.4 * B0, 0.4 * B0))
178          semilla = tuple(semilla) + (0.0,)
179
180      # multiarranque: una corrida del recocido es una muestra, no un resultado
181      corridas = sa.multiarranque(p.costo, cotas, n=n_arranques, semilla=7,
182                                  callback=cb_sa)
183      j = int(np.argmin([r["costo_pulido"] for r in corridas]))
184      x = corridas[j]["x_pulido"]
185      chi2 = corridas[j]["costo_pulido"]
186      Ds_ms = np.array([r["x_pulido"][0] for r in corridas])
```

calibracion.py · lineas 166 a 186

- **linea 166** - **las cotas**, y de donde salen. D se busca entre el 60% y el 160% de la semilla del NCC, que es una ventana generosa alrededor de una estimacion que ya es buena. σ_{rel} entre 0.02 y 0.30: menos del 2% seria un detector que no borronea nada y mas del 30% no seria un borde.
- **lineas 167 a 172** - si el modelo incluye el exceso de bajo canal, aparece un tercer parametro (λ) con su propia cota y su propia semilla. El algoritmo no cambia: busca en tres dimensiones en vez de dos.
- **linea 181** - el multiarranque, con **8** corridas por defecto.
- **lineas 183 a 186** - se elige la corrida de menor χ^2. Las otras no se tiran: su dispersion se reporta como diagnostico.

8.1 El detalle que la interfaz necesita para dibujar

Un ajuste devuelve numeros; una pestana que explica el ajuste necesita ademas el **camino**. Por eso `calibrar` acepta un diccionario opcional y lo llena con la semilla, las cotas y las N corridas completas. No entra en el resultado normal a proposito: el resultado se guarda como JSON y esto son arrays de numpy.

```
333      detalles = {}
334      r, p, _, ncc_r = calibracion.calibrar(
335          c, rama=o["rama"], metodo=o["metodo"], n_arranques=o["n_arranques"],
336          verbose=False, esp=e, plantilla=plantilla,
337          inicio_rel=o["inicio_rel"], cb_ncc=cb_ncc, cb_sa=cb_sa,
338          exceso=o["exceso"], tipo_exceso=o["tipo_exceso"],
339          inicio_ajuste_rel=o["inicio_ajuste_rel"], detalles=detalles)
```

pipeline.py · lineas 333 a 339

Con eso, la pestana **3 · Semilla y recocido** puede mostrar todo sin volver a correr nada:

panel	que muestra	que hay que mirar
1 - la tabla	de donde arranca cada corrida y donde termina	muestra la semilla del NCC, las cotas y los N arranques con su D final. Los arranques NO salen de la semilla: salen de puntos al azar adentro de las cotas.
2 - el costo por ciclo	como baja, en las N corridas a la vez	la linea gruesa es la corrida ganadora y las finas las otras. Que terminen apiladas es la evidencia practica de que el minimo es global.
3 - T y aceptacion	la mecanica del enfriamiento	al principio se acepta casi todo (la cadena pasea), al final casi nada (se comporta como un optimizador local). Si arranca en ~1 o en ~0, T_init esta mal calibrada.
4 - el camino	por donde paseo la busqueda en el plano (D, sigma)	es lo unico que distingue 'bajo en linea recta' de 'cruzo de un valle a otro', que es la unica razon por la que se usa recocido.

8.2 El resultado, con el modelo completo

Corrido tal como lo corre la interfaz -o sea con el exceso de bajo canal adentro, que agrega el tercer parametro- el ajuste de esta corrida da:

	valor	que es
D	1.2512 ± 0.0047	mV·ns por keVee: la calibracion buscada
σ_{rel}	0.0882 ± 0.0076	resolucion, como fraccion del canal del borde (20.8 % FWHM)
χ^2/ndf	1.169	257 grados de libertad
bordes	1205, 1399	donde caen, en mV·ns, los bordes Compton de Co-60
dispersion	3.3e-10	entre los 8 arranques: el optimizador no aporta incerteza

Tardo **2.2 segundos** en total, de los cuales el recocido son unas decimas: lo caro de la cadena esta antes, en leer el archivo y armar el espectro.

9. Que pasa si se mueve cada constante

La forma honesta de justificar una constante es moverla y ver que cambia. Cada fila es el mismo problema resuelto 6 veces con semillas distintas, cambiando una sola cosa respecto del default.

variante	D (mediana)	dispersion	ciclos	evaluaciones
T_init calibrada (el default)	1.2230	1.9e-09	17	1718
T_init = 2000, el numero del paper	1.2230	1.5e-08	23	2318
T_init 100 veces mas chica	1.2230	3.0e-09	20	2018
T_init 100 veces mas grande	1.2230	3.0e-10	19	1901
alfa = 0.85, enfria rapido	1.2230	2.2e-09	25	2534
alfa = 0.99, enfria lento	1.2230	1.7e-10	20	2018
paso 10 veces mas chico	1.2230	2.7e-09	17	1668
paso = el rango entero de las cotas	1.2230	5.2e-10	19	1868
10 propuestas por temperatura, no 100	1.2230	2.3e-09	18	184

La lectura: sobre este problema **ninguna de las constantes cambia el resultado**, solo cuanto tarda. Eso es lo que uno quiere de un optimizador -que el numero sea del problema y no del metodo- y tambien es la razon por la que no hay que gastar tiempo afinandolas. Lo que si cambia es el precio: enfriar mas lento o proponer poco cuesta mas evaluaciones para llegar al mismo lugar.

El caso interesante es el de las **10 propuestas por temperatura** en vez de 100: llega al mismo sitio con una decima parte del trabajo. En un paisaje con varios valles no seria asi -la cadena necesita propuestas suficientes por temperatura para explorar antes de que se enfrie- y por eso el default sigue siendo el del paper.

9.1 Lo unico que si rompe: cotas que dejan el optimo afuera

Se corrio a proposito con las cotas de D puestas en [1.35, 1.75], que no contienen el optimo (1.2230). El recocido, que recorta cada propuesta, no puede hacer otra cosa que apoyarse contra la pared. Lo llamativo es lo que pasa despues: el **pulido corre sin cotas**, se escapa de la caja y termina en D = 1.2230, el valor correcto. Salio bien por casualidad, y muestra el punto: el algoritmo no tiene forma de avisar que las cotas estaban mal, porque un numero devuelto contra la pared se ve igual que cualquier otro.

La defensa contra eso no esta en el optimizador sino aguas arriba: las cotas se derivan de la semilla del NCC -del 60 al 160 por ciento de ella- y no se eligen a mano, y el multiarranque cubre todo el rectangulo. Si el optimo estuviera contra un borde, los N arranques terminarian todos ahi y se veria en el panel del camino.

10. Preguntas que aparecen la primera vez

Si acepta cosas peores, como puede terminar en un lugar bueno?

Porque el mejor punto se guarda aparte y no se pierde nunca (`mejor_x`). La cadena pasea; el resultado es el record. Y ademas la probabilidad de aceptar algo peor se desploma a medida que baja la temperatura, asi que el paseo se apaga solo.

El resultado es aleatorio?

La busqueda si; el resultado, en la practica, no. Con la semilla fija la corrida es reproducible bit a bit, y el multiarranque mide cuanto cambia el numero entre corridas independientes: 2.7e-10, o sea 7 ordenes de magnitud menos que la barra de error.

Garantiza encontrar el minimo global?

No. El teorema que garantiza convergencia pide un enfriamiento logaritmico (T ~ c/log k), que es inaplicablemente lento. Con enfriamiento geometrico, que es lo que usa todo el mundo y lo que dice el paper, no hay garantia: hay evidencia, y es la de la seccion 7.

Por que no usar una funcion de scipy?

Porque el paper especifica este algoritmo con estas constantes, y el objetivo era replicarlo y poder auditarlo. `scipy.optimize.dual_annealing` existe y resuelve un problema parecido, pero es otro algoritmo (recocido generalizado, con saltos de Cauchy y pulido local automatico): usarlo seria comparar contra otra cosa.

Cuanto tarda?

Una corrida, 0.11 segundos (1301 evaluaciones del costo a 68 microsegundos cada una). El multiarranque de 12, 2.2 segundos. Es despreciable al lado de leer el archivo del digitalizador.

Y si el ajuste da mal?

Antes de tocar el optimizador conviene mirar las cuatro cosas de la pestana 3, en este orden: la aceptacion (si arranca en 100% o en 0%, la temperatura esta mal), la dispersion entre arranques (si es grande, hay mas de un valle), el camino (si termina contra una pared, las cotas cortan), y el paisaje (si el valle no es un pozo cerrado, las barras de error mienten).

11. Donde esta cada cosa

funcion	archivo y lineas	que hace
`armar`	modelo.py: 295-352	arma el problema: espectro + plantilla + ventana
`Problema.costo`	modelo.py: 252-259	el numero que el optimizador minimiza
`amplitud_optima`	modelo.py: 79-94	despeja la amplitud sin buscarla (VARPRO)
`temperatura_inicial`	sa.py: 69-93	calibra T_init midiendo el costo
`recocido`	sa.py: 99-206	el algoritmo: Metropolis + enfriamiento
`pulir`	sa.py: 209-221	optimizador local que baja al fondo exacto
`multiarranque`	sa.py: 224-254	N recocidos desde puntos dispersos
`errores_por_curvatura`	sa.py: 260-310	barras de error por curvatura del chi2
`calibrar`	calibracion.py: 116-255	la receta: cotas, multiarranque, resultado
`calibrar_completo`	pipeline.py: 209-358	la cadena entera, que llaman las dos interfaces
`App._fig_recocido`	gui.py: 1794-1941	los cuatro paneles de la pestana 3

Para verlo corriendo: `python analisis/sa.py` deja una figura con el algoritmo sobre esta misma corrida, `python analisis/pipeline.py co60` hace la cadena completa por consola, y `python analisis/gui.py` abre la interfaz.

Apendice. El modulo completo

Lo que sigue es `analisis/sa.py` entero -sin la demo del final- con sus comentarios y docstrings, que es donde el codigo explica sus propias decisiones. Los recuadros de las secciones anteriores son fragmentos de esto.

```
import os
import sys

import numpy as np

if __name__ == "__main__" and __package__ is None:
    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from analisis import rutas


# ---------------------------------------------------------------------------
# 1. Calibracion de la temperatura inicial
# ---------------------------------------------------------------------------
def temperatura_inicial(costo, x0, delta, aceptacion=0.8, n=60, rng=None):
    """
    T tal que un empeoramiento TIPICO se acepte con probabilidad `aceptacion`.

    Se muestrean `n` propuestas alrededor de x0, se mide cuanto empeoran en
    promedio, y se despeja T de  exp(-<DeltaE+> / T) = aceptacion:

        T = -<DeltaE+> / ln(aceptacion)

    Sin esto hay que adivinar un numero que depende de las unidades del costo, y
    adivinar mal no da error: da un recocido que no explora (T chico) o que
    nunca converge (T grande).
    """
    rng = np.random.default_rng() if rng is None else rng
    x0 = np.asarray(x0, float)
    c0 = costo(x0)
    subidas = []
    for _ in range(n):
        x = x0 + rng.uniform(-1, 1, size=x0.size) * delta
        d = costo(x) - c0
        if d > 0:
            subidas.append(d)
    if not subidas:
        return max(abs(c0) * 1e-3, 1e-9)
    return float(-np.mean(subidas) / np.log(aceptacion))


# ---------------------------------------------------------------------------
# 2. El recocido
# ---------------------------------------------------------------------------
def recocido(costo, x0, cotas, delta=None, T_init="auto", alfa=0.95,
             n_markov=100, T_min_rel=1e-6, n_estable=10, tol_rel=1e-4,
             paso_adaptativo=False, objetivo_aceptacion=0.35, semilla=None,
             max_ciclos=400):
    """
    Minimiza `costo` con recocido simulado. Devuelve un dict.

    costo  : callable(x) -> float
    x0     : punto inicial (la semilla del NCC)
    cotas  : [(min, max), ...] por parametro. Acota la busqueda a lo fisicamente
             posible; ademas de ahi sale `delta` si no se lo dan.
    delta  : escala de paso por parametro. Por defecto 0.1 * (max - min), que es
             el `0.1*d_i` del paper.
    alfa   : coeficiente de enfriamiento geometrico, T_{k+1} = alfa * T_k.
    n_markov : iteraciones por temperatura (las 100 del paper).
    T_min_rel : corte por temperatura, relativo a T_init.
    n_estable, tol_rel : corte por estabilizacion. El mejor costo tiene que
```

```
             mejorar menos de `tol_rel` (relativo) durante `n_estable` ciclos.

    Claves del resultado: x, costo, historia (una fila por ciclo de temperatura
    con T, costo actual, mejor costo, tasa de aceptacion y paso), n_eval.

    Se guarda ademas el CAMINO: `historia_x` es donde estaba la cadena al
    cerrar cada ciclo de temperatura y `historia_mejor_x` el mejor punto
    conocido hasta ese ciclo. Cuesta una fila de floats por ciclo (unos
    cientos en total) y es lo unico que permite ver despues por donde paseo el
    recocido, que es la mitad de lo que hay que mirar para creerle: `historia`
    dice que el costo bajo, `historia_x` dice si para eso hubo que cruzar de
    un valle a otro o alcanzo con bajar en linea recta.
    """
    rng = np.random.default_rng(semilla)
    x = np.asarray(x0, float).copy()
    cotas = np.asarray(cotas, float)
    lo, hi = cotas[:, 0], cotas[:, 1]
    if delta is None:
        delta = 0.1 * (hi - lo)
    delta = np.asarray(delta, float).copy()

    T = (temperatura_inicial(costo, x, delta, rng=rng)
         if T_init == "auto" else float(T_init))
    T0 = T
    escala = 1.0

    c = costo(x)
    c_inicial = c
    mejor_x, mejor_c = x.copy(), c
    historia = []
    historia_x = []
    historia_mejor_x = []
    estables = 0
    n_eval = 1

    for ciclo in range(max_ciclos):
        aceptados = 0
        for _ in range(n_markov):
            prop = x + rng.uniform(-1, 1, size=x.size) * delta * escala
            prop = np.clip(prop, lo, hi)
            cp = costo(prop)
            n_eval += 1
            dE = cp - c
            # Metropolis: si mejora se acepta siempre; si empeora, con exp(-dE/T).
            if dE <= 0 or rng.random() < np.exp(-dE / max(T, 1e-300)):
                x, c = prop, cp
                aceptados += 1
                if c < mejor_c:
                    mejor_x, mejor_c = x.copy(), c
        tasa = aceptados / n_markov
        historia.append((T, c, mejor_c, tasa, escala))
        historia_x.append(x.copy())
        historia_mejor_x.append(mejor_x.copy())

        # paso adaptativo [fuera del paper, apagado por defecto]: si se acepta
        # mucho, el paso es demasiado timido y conviene agrandarlo; si se acepta
        # poco, achicarlo. Sobre ESTE problema no cambia nada (verificacion 4),
        # asi que no se usa; se deja para modelos con mas parametros.
        # El objetivo 0.35 no es un numero canonico: los valores citados en la
        # literatura de MUESTREO por Metropolis son 0.44 (1 dimension) y 0.234
        # (asintotico en muchas). Barrido entre 0.15 y 0.60, D no se mueve mas
        # de 0.011% (verificacion 5).
        if paso_adaptativo:
            if tasa > objetivo_aceptacion + 0.1:
                escala *= 1.25
            elif tasa < objetivo_aceptacion - 0.1:
                escala *= 0.8
            escala = float(np.clip(escala, 1e-4, 20.0))

        # corte por estabilizacion (relativo, no absoluto)
        if len(historia) > 1:
            prev = historia[-2][2]
            if abs(prev - mejor_c) <= tol_rel * max(abs(prev), 1e-12):
                estables += 1
```

```
            else:
                estables = 0
            if estables >= n_estable:
                break

        T *= alfa
        if T < T_min_rel * T0:
            break

    return dict(x=mejor_x, costo=mejor_c, historia=np.array(historia),
                historia_x=np.array(historia_x),
                historia_mejor_x=np.array(historia_mejor_x),
                x_inicial=np.asarray(x0, float).copy(),
                costo_inicial=float(c_inicial), delta=delta.copy(),
                n_eval=n_eval, T_init=T0, ciclos=len(historia),
                motivo=("estabilizacion" if estables >= n_estable
                        else "temperatura minima"))


def pulir(costo, x, cotas=None):
    """
    Optimizador local desde donde dejo el recocido.

    El recocido encuentra el valle bueno pero se detiene con la precision del
    ultimo paso, que no es cero. El pulido baja al fondo exacto. Los dos hacen
    falta: el recocido sin pulido da el 3er decimal, el pulido sin recocido da
    el valle equivocado.
    """
    from scipy.optimize import minimize
    r = minimize(costo, np.asarray(x, float), method="Nelder-Mead",
                 options=dict(xatol=1e-7, fatol=1e-4, maxiter=4000))
    return r.x, float(r.fun)


def multiarranque(costo, cotas, n=8, x0=None, dispersion=0.25, semilla=0,
                  callback=None, **kw):
    """
    Corre `n` recocidos desde puntos iniciales DISPERSOS y devuelve todos.

    El recocido es estocastico: una sola corrida no es un resultado, es una
    muestra. Que N arranques independientes y dispersos terminen en el mismo
    punto es la evidencia practica de que ese punto es el minimo global (no una
    demostracion: ver el PDF).
    """
    rng = np.random.default_rng(semilla)
    cotas = np.asarray(cotas, float)
    salidas = []
    for i in range(n):
        if callback is not None:
            # el avance se cuenta por ARRANQUE y no por ciclo de temperatura:
            # cuantos ciclos va a hacer cada recocido no se sabe de antemano
            # (corta por estabilizacion), asi que inventar ese denominador daria
            # una barra que retrocede.
            callback(i / n, "arranque %d de %d" % (i + 1, n))
        if x0 is None:
            xi = rng.uniform(cotas[:, 0], cotas[:, 1])
        else:
            xi = np.asarray(x0, float) * (1 + dispersion *
                                          rng.uniform(-1, 1, size=len(x0)))
            xi = np.clip(xi, cotas[:, 0], cotas[:, 1])
        r = recocido(costo, xi, cotas, semilla=int(rng.integers(1 << 30)), **kw)
        r["x_pulido"], r["costo_pulido"] = pulir(costo, r["x"])
        r["x0"] = xi
        salidas.append(r)
    return salidas


# ---------------------------------------------------------------------------
# 3. Como saber si el optimo es bueno
# ---------------------------------------------------------------------------
def errores_por_curvatura(costo, x, paso_rel=1e-3, delta_chi2=1.0, paso=None):
    """
    Barras de error a partir de la CURVATURA del costo en el minimo.

```

```
    Para un chi2, la matriz de covarianza es `2 * H^-1` con H el hessiano de
    chi2/2, o sea `C = 2 * (d2chi2/dxdx)^-1`. El error de cada parametro es la
    raiz de la diagonal, y corresponde al contorno `chi2_min + 1`.

    Vale solo si el minimo es aproximadamente cuadratico, que es justamente lo
    que hay que mirar en el mapa del paisaje antes de creerle.

    `paso` permite dar el paso de derivacion de cada parametro a mano, en vez
    del `paso_rel * |x|` de siempre. Hace falta cuando la escala natural de un
    parametro no es su propio valor. El caso que lo obligo: la ordenada c0 del
    mapa canal-energia. La plantilla se lleva a canal con `np.interp`, o sea
    que es lineal a trozos sobre la grilla del histograma; con c0 = -163 el
    paso relativo da 0.16 mV*ns, un 3% de un bin de 5.4, y ahi la "curvatura"
    que se mide es el quiebre entre dos trozos y no la del chi2. Daba 21 sigma
    donde el cociente de verosimilitud da 2.1. Para ese parametro el paso tiene
    que ser del orden de UN BIN.

    Control: para D, cuya escala si es su propio valor, el paso no cambia nada
    (err_D = 0.0040 con paso_rel 1e-2 y 0.0044 con 3e-4) y coincide con el
    perfil real de Delta_chi2 = 1, que da 0.0040.
    """
    x = np.asarray(x, float)
    n = x.size
    if paso is not None:
        h = np.abs(np.asarray(paso, float))
    else:
        h = np.maximum(np.abs(x) * paso_rel, 1e-9)
    H = np.empty((n, n))
    c0 = costo(x)
    for i in range(n):
        for j in range(n):
            xp, xm = x.copy(), x.copy()
            if i == j:
                xp[i] += h[i]; xm[i] -= h[i]
                H[i, i] = (costo(xp) - 2 * c0 + costo(xm)) / h[i] ** 2
            else:
                a = x.copy(); a[i] += h[i]; a[j] += h[j]
                b = x.copy(); b[i] += h[i]; b[j] -= h[j]
                cc = x.copy(); cc[i] -= h[i]; cc[j] += h[j]
                d = x.copy(); d[i] -= h[i]; d[j] -= h[j]
                H[i, j] = (costo(a) - costo(b) - costo(cc) + costo(d)) / (4 * h[i] * h[j])
    try:
        C = 2.0 * np.linalg.inv(H)
        err = np.sqrt(np.maximum(np.diag(C), 0.0)) * np.sqrt(delta_chi2)
        return err, C
    except np.linalg.LinAlgError:
        return np.full(n, np.nan), np.full((n, n), np.nan)
```

sa.py · lineas 55 a 310 · el modulo, sin la demo